

Raising Exceptions vs. Returning Error Codes

In Python, the meanings of the terms *exception* and *error* are roughly the same: an exceptional circumstance in your program that usually indicates a problem. Exceptions became popular as a programming language feature in the 1980s and 1990s with C++ and Java. They replaced the use of *error codes*, which are values returned from functions to indicate a problem. The benefit of exceptions is that return values are only related to the function's purpose instead of also indicating the presence of errors.

Error codes can also cause issues in your program. For example, Python's `find()` string method normally returns the index where it found a substring, and if it's unable to find it, it returns `-1` as an error code. But because we can also use `-1` to specify the index from the end of a string, inadvertently using `-1` as an error code might introduce a bug. Enter the following in the interactive shell to see how this works.

```
>>> print('Letters after b in "Albert":', 'Albert'['Albert'.find('b') + 1:])
Letters after b in "Albert": ert
>>> print('Letters after x in "Albert":', 'Albert'['Albert'.find('x') + 1:])
Letters after x in "Albert": Albert
```

The `'Albert'.find('x')` part of the code evaluates to the error code `-1`. That makes the expression `'Albert'['Albert'.find('x') + 1:]` evaluate to `'Albert'[-1 + 1:]`, which further evaluates to `'Albert'[0:]` and then to `'Albert'`. Obviously, this isn't the code's intended behavior. Calling `index()` instead of `find()`, as in `'Albert'['Albert'.index('x') + 1:]`, would have raised an exception, making the problem obvious and unignorable.

The `index()` string method, on the other hand, raises a `ValueError` exception if it's unable to find a substring. If you don't handle this exception, it will crash the program—behavior that is often preferable to not noticing the error.

The names of exception classes often end with “Error” when the exception indicates an actual error, such as `ValueError`, `NameError`, or `SyntaxError`. Exception classes that represent exceptional cases that aren't necessarily errors include `StopIteration`, `KeyboardInterrupt`, or `SystemExit`.